

AI Methods for Research Workers

Scott Cunningham

Baylor University · Visiting, Harvard Government 2025–26

MIT · Cinco de Mayo

Basque: Maiatzaren bosta [mai-AH-tsah-ren BOH-stah]

Who am I?

Scott Cunningham. Ben H. Williams Professor of Economics, Baylor University.
Visiting Professor of Government, Harvard — wrapping up the year.

Research interests — mostly applied empirical work:

- ▶ Corrections
- ▶ Suicide
- ▶ Causal inference
- ▶ Drug policy
- ▶ Relationship markets
- ▶ Sex work
- ▶ Minimum wages
- ▶ AI agents

Today: not a tour of prompts. A framework for using agents to produce and verify research without pretending the risks are fake.

Theatrical stunt

The setup

- ▶ drop one paper into a clean folder
- ▶ run `/split-pdf` to digest it
- ▶ run `/beautiful_deck` with a careful prompt
- ▶ ask Claude to implement the estimator from scratch
- ▶ verify against a known empirical setting

The paper

Abadie (2003). κ -weighting for the LATE.

The verification

Card (1995) college-in-the-county returns to schooling — a setting where the IV LATE is well understood.

One paper. One workflow. Production is the demo. Verification is the standard.

1

The Conceptual Framework

Trust Nothing and Verify Everything

In April 2026, Sullivan & Cromwell apologized to a federal judge after a filing contained AI-generated errors, including fabricated legal citations.

The striking part is that the firm's AI training already included the rule:

**Trust Nothing
Verify Everything**

The rule was right. The workflow still failed.

Production and verification are different tasks

Imagine research as a series of folders on your computer, full of files you manipulate in two distinct tasks:

Production

Generating drafts, code, tables, simulations, literature summaries, slides, and alternative versions.

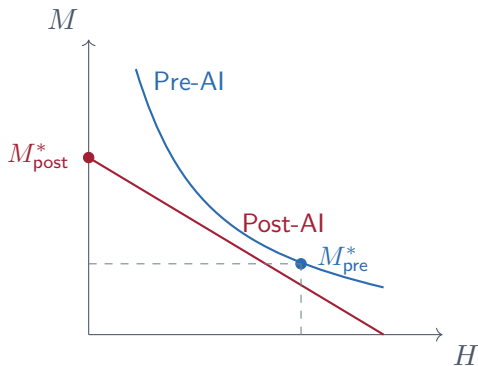
Verification

Knowing whether the output is true, faithful, correctly estimated, and worth believing.

Historically these were bundled. AI agents are pulling them apart.

Comparative advantage in production varies across researchers and shifts over time. But the pipeline from idea to scientific object — papers, patents, grants — is being reshaped by agents. Anyone who has worked intensively with them can see this.

Shape of the cognitive-output isoquant in machine and human time



Pre-AI: human time and machine help were complements. Post-AI: machine time substitutes for human time in production.

$$Q = f(H, M)$$

- ▶ H = human time
- ▶ M = machine time

The question is not whether to use M . It is how much H you intentionally keep.

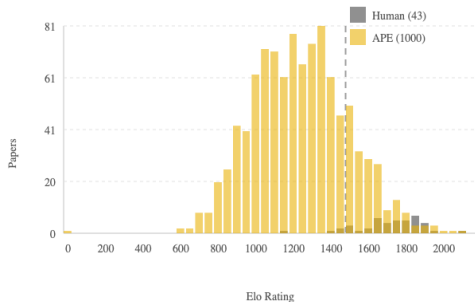
Autonomous research was already here

The Social Catalyst Lab's **Project APE** has produced **1,000+ autonomously generated empirical policy papers**.

Agents that:

- ▶ pull public data
- ▶ run empirical designs
- ▶ write complete papers

Research automation is no longer hypothetical.



ELO tournament: APE papers vs. human papers.

I did it too — the prompt

I have a manuscript due to a conference tonight. I forgot what I proposed.
My research interests are: drug policy, abortion, suicide, corrections,
causal inference, sex work.

Come up with the research question. It must be substantive --
a question of moral and economic importance ...

Find all the appropriate data sources. Pull them into local folders.
Arrange everything into a replicable pipeline ...

Choose an identification strategy, a population, an estimand, an
estimator. Estimate it. Run robustness tests ...

Report results in rhetorically appropriate exhibits -- beautiful
figures, beautiful tables ...

Write the manuscript in the voice and style of Martin Weitzman, my
favorite economic writer. Each sentence should feel belabored over --
scientifically precise, morally heavy, just barely restrained ...

22 pages, double-spaced. The first sentence must immediately capture
the reader's attention and make them want to read the second sentence,
then the first paragraph, and so on ...

Do not overreach. Tell the truth.

Three times — like laundering money

What I actually did was nowhere near fully automated.

When the manuscript came back, I dropped it into **Refine.ink**, downloaded the markdown report, asked Claude to address every comment, took the new draft, dropped it into Refine.ink again.

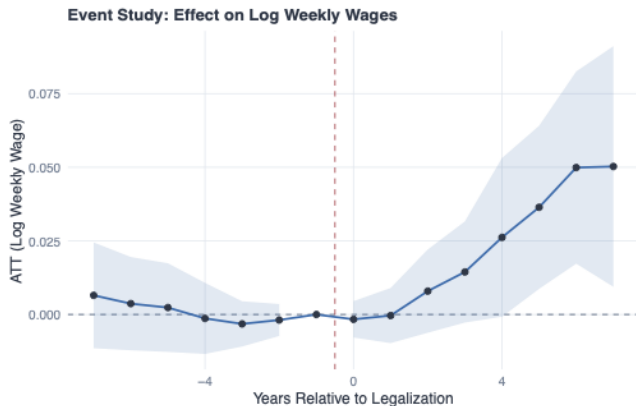
Three times. Like I was laundering money.

Then /referee2 reran the pipeline in Python, R, and Stata to confirm no implementation errors. /tikz audited every figure for visible collisions.

And then I opened the paper to see what Claude produced.

It was a study of cannabis policy and labor markets.

The event study — AI-generated, then laundered

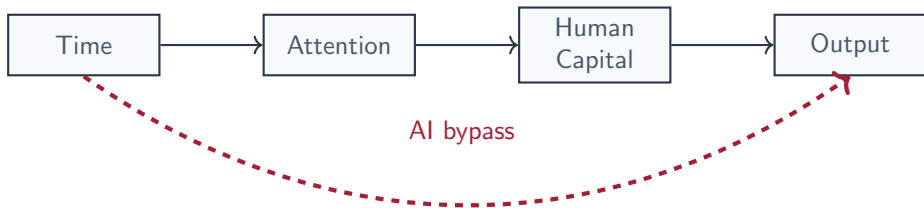


Cannabis policy and wages. Callaway–Sant’Anna event study. Rising wages following reform.

Deskilling is real, but not deterministic

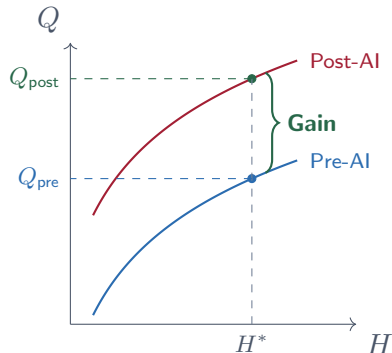
The gains are real. People will use them.

The risk is also real: if agents let us bypass attention, we may stop building the human capital needed to judge the output.



This is a design problem, not destiny. Build verification workflows that keep judgment in the loop.

Case I: keep effort constant, take the output gain

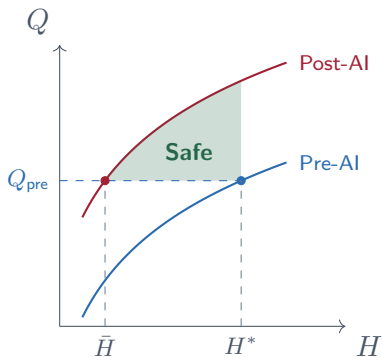


Keep H^* the same. Do more and better work in the same hours.

- Biggest gains in output.
- Human capital preserved.

This requires us not to take all the time savings.

Case II: reclaim time, but stay safe



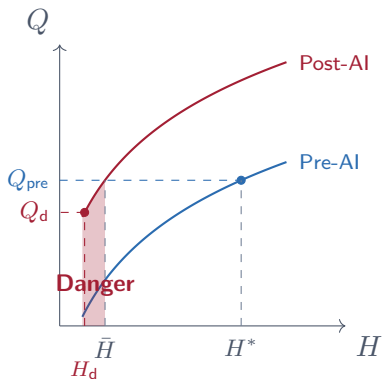
Cut hours, but stay in the green.

As long as $H > \bar{H}$, output remains above baseline.

- Moderate time savings.
- Still a net gain in output.

Some slack is fine. The curve shifted up enough to absorb it.

Case III: the depreciation trap



The red zone. Skill depreciation without the effort to maintain $H > \bar{H}$ means $Q_{\text{post}} < Q_{\text{pre}}$.

- Worse off than before AI.

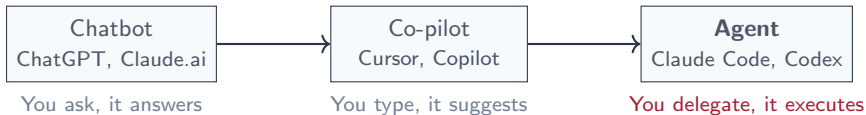
The point is not to avoid time savings.
The point is to be intentional.

2

Setup and Basics

Agents are different because they act

A spectrum of AI use, from passive to autonomous:



An agent reads files, writes code, runs it, reads the output, and decides what to do next across many steps.

What the agent actually does

An agent is not just a chatbot with better prose. It acts on your project.

<code>ls data/</code>	list files
<code>rg "minimum wage" .</code>	search text and code
<code>Rscript estimate.R</code>	run an R script
<code>python audit.py</code>	run a Python script
<code>pdflatex paper.tex</code>	compile a paper or deck
<code>git diff</code>	inspect changes

It reads files, writes files, runs commands, sees errors, revises, and tries again.

Reasoning models and agents are merging

Reasoning LLMs

o1, o3, DeepSeek R1, Claude extended thinking

- ▶ Trained to deliberate before answering
- ▶ Pure inference: no action in the world
- ▶ Strong at math, code, logic puzzles

AI agents

ReAct, tool use, Claude Code, Cursor, Codex

- ▶ LLM + tools: read, write, run, search
- ▶ Loop: act, observe, decide, repeat
- ▶ Strong at long, multi-step tasks

2025–26: the merger. Frontier systems increasingly *think* and *act*: plan a task, execute steps, check output, and revise.

Claude Code began as a terminal agent

Boris Cherny joined Anthropic in 2024. Late that year he gave Claude direct access to his terminal and asked it to fix his Spotify playlists.

Claude wrote API calls, read its own output, handled errors, and finished the job autonomously. *That was the proof-of-concept.*

- ▶ **Late 2024.** Internal prototype.
- ▶ **Feb. 24, 2025.** Research preview with Claude 3.7 Sonnet.
- ▶ **May 2025.** General availability.
- ▶ **Nov. 25, 2025.** Claude Code lands inside the desktop app.

Desktop integration of Claude Code is to AI agents what ChatGPT was to GPT — product, not capability, is probably the unspoken hero of this revolution.

Install Claude Code by asking Claude to install it

You have to get Claude Code onto your machine somehow. The install command is the first place phishing finds you.

The trojan-horse problem

- ▶ cloned sites imitating `claude.ai`, GitHub, package docs
- ▶ “copy” buttons that hide a different command than the one shown
- ▶ pasting an unfamiliar install command is voluntary malware

The defensive habit

- ▶ open the desktop app (or another agent you trust)
- ▶ give it the URL you were about to paste from
- ▶ have it inspect domain, repo, command, and explain
- ▶ run nothing until the explanation matches what you expect

Anthropic is currently doing better than most at not getting cloned. That will not last. Build the habit before the arms race catches up.

Claude is not one interface

The terminal is where Claude Code began. It is still the cleanest place to work with project folders, code, data, and reproducible files.

CLI / terminal

- ▶ Project directories
- ▶ File edits and code runs
- ▶ Repeatable workflows

Desktop / Cowork

- ▶ Conversational work
- ▶ Shared context
- ▶ Faster collaboration loops

Read Chris Cornwell's Claude academic guide:

<https://cornwl.github.io/files/claude-academic-guide.html>

Explain which parts apply to my project.

Help me choose CLI, Desktop/Cowork, or both.

Agents are a general-purpose technology

The same agent runs my Callaway–Sant’Anna replication *and* grabs a file off the internet when I need it.

Trivial uses

- ▶ push to my repos
- ▶ edit my website
- ▶ update my CV
- ▶ fetch from the internet

Sophisticated uses

- ▶ replication audits
- ▶ bibliography checks
- ▶ package comparisons
- ▶ many-agent experiments

Desktop fits the trivial. The CLI fits the sophisticated. The CLI feels like DOS or Unix — bare — which is why it goes fast on work that benefits from less ceremony.

Most of the daily learning about how to work with agents happens at the trivial end.

Three operating modes

Mode	What it does	Where
Plan	Read-only. Searches, reads, proposes. No state changes.	CLI & Desktop
Action	Edits, runs, writes. Each risky tool call requests approval.	CLI & Desktop
<code>--dangerously-skip-permissions</code>	Edits, runs, writes — no approvals. The agent runs free.	CLI only

Why the third mode matters: the agent goes fast. Launch a long task, walk away, come back to a finished result. Not available in the desktop app. You have given up the per-tool checkpoint, so the safety question is no longer the flag — it is the blast radius of the folder.

CLAUDE.md is the project constitution

CLAUDE.md is the file Claude reads to understand how this project works.

It can record:

- ▶ project purpose and current state
- ▶ folder structure and important files
- ▶ coding, writing, and style rules
- ▶ how to compile, test, audit, and review
- ▶ what the agent should never do without asking

In a project folder:

claude

Read this repo and draft a CLAUDE.md for how we should work.

Ask me before writing the final version.

Start with CLAUDE.md; add skills only when a task repeats.

Skills are reusable research routines

CLAUDE.md tells the agent how this project works. A **skill** tells the agent how to do a recurring task. The ones I made because I do them constantly:

Production

- ▶ /beautiful_deck
- ▶ /tikz
- ▶ /split-pdf

Verification

- ▶ /referee2
- ▶ /blindspot

Literature audit

- ▶ /bibcheck

This is how one-off prompting becomes a workflow.

PDFs are still a bottleneck

It feels like a PDF contains text. Often it does, but not in the way a markdown file contains text.

PDFs are closer to layout instructions: glyphs, coordinates, columns, figures, footnotes, headers, OCR artifacts, and broken reading order.

For small papers, agents can often manage this. For giant PDFs, scans, appendices, or books:

- ▶ the chat may slow down or die
- ▶ the context may fill with layout debris
- ▶ the agent may lose the thread
- ▶ your work can disappear with the session

Do not assume “it can read PDFs” means “it can reliably reason over any PDF.”

MixtapeTools: /split-pdf

/split-pdf is my answer to giant PDFs.

Instead of asking one agent to swallow the whole file, the workflow decomposes the document:

1. split the PDF into smaller chunks
2. send chunks to parallel agents
3. write markdown summaries for each chunk
4. run a final synthesis agent over the summaries
5. save a definitive markdown representation

The goal is not just reading. The goal is a durable intermediate object the next agent can inspect.

3

Production

Use agents when the work is valuable, slow, and checkable

My view: delegate to agents when a task satisfies all four conditions.

1. **The output is highly valuable.**

The stakes justify the setup cost.

2. **The task is time-intensive for humans.**

Hours or days of skilled effort, not minutes.

3. **The probability humans do it well is low.**

Difficult to execute at high quality consistently.

4. **The output can be verified.**

You can inspect, compare, rerun, or source-ground it.

Coding is the canonical example, but this also describes literature review, replication, deck creation, and referee-style review.

Decks are the gateway drug

Slides are valuable. When I graduated in 2007, all of us shared Imbens and Wooldridge's NBER lectures. They came out before *Mostly Harmless*. No textbook — just slides. They were the bible for an entire cohort.

Hard to make well. Extremely time-consuming, sometimes infinitely so. Especially when they require TikZ — a powerful, dense graphics package Claude knows backwards and forwards.

I usually make bad slides because I cannot stop tinkering long enough to do what I actually like doing: thinking about how the talk should land.

Verifiable in front of you. Good or bad, the slide is right there. You can see it.

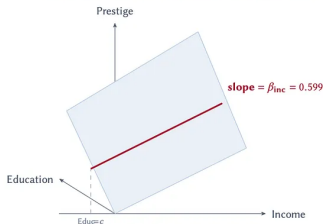
Culturally different from writing. Publishers give you slides. We share each other's slides. We do not even know how to cite each other's slides.

So slides are the place where delegating to an agent feels lowest-stakes and highest-leverage simultaneously. Start there.

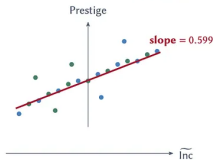
The FWL diagram I could not see in my head

The BLP plane slice and FWL both recover slope $\beta_{inc} = 0.599$ — from the same partial variation

Long BLP plane: fix Education, vary Income



FWL: Prestige vs $\widetilde{\text{Inc}}$



Both red lines have slope $\beta_{inc} = 0.599$ — the BLP plane slice and FWL are the same object

Last week: fixing Education at any value c gives a line through the BLP plane with slope β_{inc} . FWL extracts that slope from the 2D scatter by removing Education's variation.

`/beautiful_deck`

`/beautiful_deck` is built around a simple idea: a deck is not a document dump. It is rhetoric under time pressure.

`https://github.com/scunning1975/mixtape-tools`

Give Claude the repo link and say:

Read the MixtapeTools repo.

Install or adapt the `beautiful_deck` skill for this project.

Use the `rhetoric-of-decks` essay as the standard.

This is production, but the standard is rhetorical: pathos, ethos, logos, low density, high impact.

Decks are rhetoric, not document dumps

/beautiful_deck sits on top of a short essay — *The Rhetoric of Decks*.

The standard is not coverage. It is persuasion under time pressure:

- ▶ titles are assertions, not labels
- ▶ one idea per slide
- ▶ ethos, pathos, logos — in that order, every time
- ▶ beauty is function: every element earns its place
- ▶ compile clean. zero warnings. always.

The skill is not building slides. It is teaching the agent what a good talk looks like.

`/beautiful_deck` **compiles clean.** `/tikz` **sees what it cannot.**

`/beautiful_deck` loops on overfull boxes, font warnings, undefined references — the things `pdflatex` catches. It will not stop until they are gone.

But `pdflatex` **never warns** that a label sits on a Bezier curve, a legend overlaps the plot, or an axis title is clipped at the figure edge. `ggsave` does not refuse to write the PNG. The producing agent declares the figure done.

LLMs have poor spatial reasoning over rendered output. Eyeballing is unreliable. So `/tikz` replaces it with two mechanisms:

- ▶ **Math mode:** Bezier-curve geometry on the source — chord length, bend angle, label safe zone
- ▶ **Visual mode:** multimodal vision on the rendered PNG, looking for the same three errors

Compile-clean is necessary. Visually clean is what makes the figure beautiful.

Example: clean a chaotic project folder

Many research projects are not clean pipelines. They are sediment.

- ▶ old scripts with unclear order
- ▶ duplicated data and output files
- ▶ tables whose source is no longer obvious
- ▶ folders organized by memory, not documentation
- ▶ projects that become emotionally costly to reopen

A safe first agent task: inventory, map, and propose. Do not move or rewrite yet.

Example: my Downloads folder

This is not a research project. It is where research materials, teaching files, installers, images, and forgotten data extracts accumulated.

Total files	31,014
Top-level items	456
Total size	75 GB
Date range	April 2012 → today
Top-level folders	28

Top-level mix: 82 CSVs, 53 PDFs, 51 HEICs, 34 Stata .do files, 19 forgotten .dmg installers, 18 .tex, 18 .rmd, 15 R scripts, 15 MP4s, 15 .dta.

456 top-level items is already too many for Finder. So the first job is not cleanup. It is diagnosis.

The mess has a shape

The single largest item is a 24 GB folder named 154561-V1: probably an IPUMS extract I unzipped years ago and forgot.

- ▶ 9.3 GB of teaching folders
- ▶ 9.5 GB in one `cps_00025.dat` file
- ▶ five MP4 episode files at 1.5–2.8 GB each
- ▶ the five biggest items account for about 50 GB of 75 GB

This is exactly the kind of task where an agent can make the invisible structure visible before anyone touches a file.

Two folder workflows

Clean an old project

Read this folder. Do not modify files.
Map data, code, outputs, and pipeline
order.

Flag inconsistencies.

Start correctly

`github.com/scunning1975/mixtapetools`
`/newproject my-project`

Creates a standard scaffold: CLAUDE.md,
README.md, code/, data/, output/,
documents/, decks/, notes/, and
progress_logs/.

The shared principle: make structure visible before asking for judgment.

Using many agents to understand agents

If automated papers are going to keep coming, I wanted to understand how the agents behaved — under strict constraints, then under slightly looser ones.

- ▶ how do they behave under tight constraints
- ▶ how do they behave under looser discretion
- ▶ what changes when context is selective but true
- ▶ what happens when warnings conflict with available choices

I chose the minimum wage. Divisive literature, lots of experiments, lots of data — and potentially a lot of discretion.

Three waves of the minimum-wage agent experiment

- Wave 1. Agents were bound to Callaway–Sant’Anna. The context moved rhetoric more than estimates.
- Wave 2. TWFE, BJS, and CS were all on the menu. Negative-context agents shifted toward TWFE, often with continuous treatment and longer panels.
- Wave 3. Replaced Meer–West in the negative prime. Continuous treatment disappeared, but the TWFE shift remained.

Even true, innocuous-looking citations can nudge agents toward a biased estimator when the workflow leaves that path open.

Cheap agents let us experiment on the research process itself

In one project, I gave hundreds of agents the same minimum-wage data and varied their literature context and estimator constraints.

The question was not whether the agents were “right.”

The question was behavioral:

- ▶ What choices do agents make when the task is underspecified?
- ▶ Which constraints bind?
- ▶ Which warnings are ignored?
- ▶ When does context affect estimator choice?

Once agents are cheap, you can run experiments on the research process itself.

Constraints discipline agents better than advice

The practical lesson was simple.

Written methodology guidance was not enough. When an estimator menu opened, agents took available paths that changed the estimand.

Package-level constraints worked better:

- ▶ the wrong comparison group is unavailable
- ▶ continuous treatment errors out
- ▶ no clean controls means no estimate

If a choice is dangerous, do not merely warn the agent. Remove or bind the choice.

When agents fail, what failed?

It is too simple to call every bad output a hallucination.

Production failed

The agent made the wrong thing: bad code, bad draft, bad search, bad translation.

Verification failed

The output looked plausible, but nobody checked the right object.

Direction failed

The human gave the wrong task, context, constraints, or stopping rule.

My prior: workflow failures explain more agent failures than people think.

4

Verification

Verification is the new bottleneck

Agents make production cheap enough to separate from verification.



The bottleneck moves. The hard part is no longer producing first drafts, first regressions, or first summaries.

The hard part is designing verification systems that survive cheap production.

Verification systems have design ingredients

Do not ask whether one output “looks right.” Ask how the workflow makes errors visible.

Useful ingredients:

- ▶ independent starts
- ▶ narrow assignments
- ▶ adversarial roles
- ▶ cross-language checks
- ▶ forced comparison against source material

The unit of trust is the workflow, not the single answer.

Audits are one way to buy verification

Four empirical-research audits I use as verification tools:

1. **Literature audit** — one agent per cited paper, checking accuracy against source text.
2. **Rhetoric audit** — one agent turns a paper into a talk; another judges whether the story is faithful.
3. **Behavior audit** — many agents become experimental units, revealing how agents behave under constraints.
4. **Implementation audit** — independent agents re-run the same analysis across languages or packages.

The goal is not audit for its own sake. The goal is confidence that the claim, code, citation, or estimate is right.

Skills make audits repeatable

An audit becomes more valuable when it can be run again.

Public starting points:

- ▶ **Hugo Sant'Anna, Clo-Author:** hsantanna.org/clo-author/
- ▶ **Pedro Sant'Anna workflow:** psantanna.com/claude-code-my-workflow/
- ▶ **MixtapeTools:** referee reports, deck building, verification routines.

A skill turns a good one-off instruction into a procedure other agents can reuse.

Borrow templates; use what you actually understand

Two public starter templates, both worth a look:

- ▶ **Clo-Author** (Hugo Sant'Anna) — per-paper pipeline: lit discovery, paper review, methods checks, writing, submission. hsantanna.org/clo-author/
- ▶ **Claude Code workflow** (Pedro Sant'Anna) — per-researcher OS: slides, papers, replication, quality gates. psantanna.com/claude-code-my-workflow/

What I actually do day to day:

1. Drop the paper into **Refine.ink**, download the markdown report, walk through revisions with Claude. Pricey, but a desk reject is more expensive.
2. Use my own /referee2 constantly. Not because it's better — because I built it, I understand it, and it conforms to my objectives.

Borrow the template; build the routine. The thing you actually use needs to be the thing you understand.

MixtapeTools: /referee2

/referee2 is my verification routine for work I do not want the original producing agent to grade.

The core idea is independence:

- ▶ a fresh agent reads the artifact cold
- ▶ it checks code, tables, claims, or slides against source material
- ▶ for code, it can reimplement in another language
- ▶ for decks, it can audit rhetoric, density, compilation, and fidelity

The original agent can explain itself. A fresh verifier is more likely to catch what the first one normalized.

MixtapeTools: /bibcheck

I built /bibcheck to verify my own bibliography was accurate. The goal is to use agents for the lowest-hanging-fruit verification tasks — the ones that are tedious by hand and exactly the kind of thing an agent can do well at scale.

Why narrow. A single agent asked to audit 80 citations drifts. The first 15 get careful treatment; the next 65 get pattern-matched. Splitting the work — one agent per entry — gives each agent full attention budget on a small task.

- ▶ one agent per .bib entry, run in parallel
- ▶ each verifies DOI/URL and cross-checks that every field belongs to the same paper
- ▶ catches the silent error: title of paper A paired with authors of paper B

The bottleneck moves from agent attention to orchestration. Cheap parallel agents handle orchestration well.

Two modes: per-citation, per-field

Per-citation (default)

```
/bibcheck refs.bib
```

One agent per entry. Each fully audits one citation: locate the canonical source, confirm every field belongs to the same paper, emit a corrected entry if needed.

Catches mixed-up entries.

Per-field

```
/bibcheck --by-field refs.bib
```

One specialist per field (title, year, journal, ...) reads the whole bibliography but only checks one thing. Each specialist runs as an isolated `claude -p` subprocess — no shared context.

Catches systematic transcription errors.

Outputs: `bibcheck_report.md` and a drop-in `corrected.bib`. Source file is never auto-overwritten.

The two modes are complements. If a manuscript cannot afford a citation error, run both.

Principles of an audit

An audit is not asking the model whether something seems right.

It is a structured workflow with numbered steps:

1. **Decompose:** one claim, file, table, or citation at a time.
2. **Isolate:** assign each unit to a fresh agent with fresh context.
3. **Ground:** compare against the actual paper, code, data, or log.
4. **Trace:** record exactly what the agent read, checked, and concluded.
5. **Report:** return tables, excerpts, file paths, and line references.
6. **Adjudicate:** the human decides what the evidence means.

The audit should show what the agents did clearly enough that you can judge the disagreement.

Save the work before the context dies

Long agent sessions fail. The antidote is to make the project remember what happened.

Write a progress log in `progress_logs/`.

Summarize everything we have done since the last update:

- decisions made
- files read
- files changed
- remaining questions
- next safe steps

Then a fresh agent can re-enter the folder, read the markdown logs, and continue without relying on the vanished chat.

Progress logs turn fragile conversations into durable project memory.

Could I replicate the PNAS labeling for \$11?

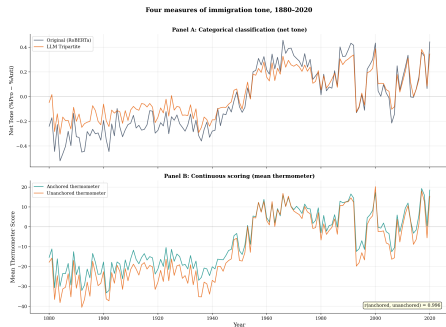
I wanted to see if I could replicate Card, Boustan and Abramitzky (PNAS 2022) on 140 years of US Congressional speeches — but using OpenAI zero-shot instead of their fine-tuned RoBERTa.

What they did. Trained RoBERTa on $\sim 7,500$ speeches hand-labeled by roughly a dozen Princeton students. Probably tens of thousands of dollars and weeks-to-months of human time. *Cost-prohibitive for most resource-constrained researchers.*

What I did. Wrapped each segment in a short prompt, zero-shotted through the OpenAI Batch API on gpt-4o-mini. The hard part is getting the JSON formatting right — which Claude is very good at handling.

Cost. About \$11 and one hour. Then I had Claude produce the comparison against their published labels.

Independent labeling tracks Card–Boustan–Abramitzky across 140 years



Net-tone trajectory: zero-shot gpt-4o-mini on 305,000 Congressional speech segments vs.

Card-Boustan-Abramitzky's RoBERTa (PNAS 2022). No fine-tuning. No human annotators.

This is what verification looks like when the production process is replaced.

Independent implementations turn disagreement into evidence

Replicating analysis in a different language is not about new data. It is about verifying the original implementation.

The logic: syntax errors are probabilistic and language-specific.

- ▶ A merge error in R will not reproduce identically in Stata.
- ▶ A package-specific fallback in Python will not usually mimic one in R.
- ▶ Agreement rules out a wide class of bugs; disagreement tells you where to look.

This is the principle behind `/referee2`: a cold-start agent reimplements the analysis independently.

Verification works when errors are imperfectly correlated

No checker is the oracle. The point is to compare independent error processes.

Checker 1	Checker 2	What disagreement reveals
Agent	fresh agent	prompt- or context-specific errors
Agent	human RA	machine vs. human blind spots
R code	Stata/Python code	implementation-specific bugs
Manuscript	source paper	citation and interpretation errors

The standard is zero avoidable errors. One pass is not enough.

Coefficient extraction at scale

I gave many agents a corpus of ~ 600 **APE** papers (Social Catalyst Lab's autonomously generated empirical-policy corpus) and asked them to extract every regression coefficient and standard error.

- ▶ agents read PDFs, located tables, emitted structured rows
- ▶ I built my own t-statistic distribution from the extracted data
- ▶ I planned to run Brodeur-style p-hacking diagnostics on top

Production was the means. The point of doing it was to verify a claim.

The first pass looked like p-hacking

The Brodeur ratio came in around **1.52**. Excess mass piled up at $|t| = 2$.

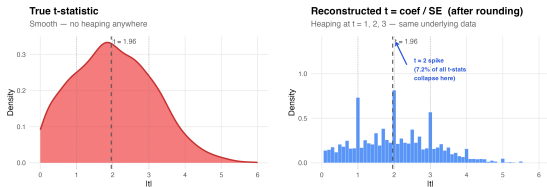
Initial reading: AI-generated empirical papers carry the same publication-bias signature as the human literature.

This was the wrong reading.

Non-random heaping I didn't see

Rounding Turns a Smooth Distribution into a Heaped One

Both panels show the SAME 3,500 simulated regressions. Left: the true $|t|$ -statistic. Right: what you recover when you extract rounded coefficients and SEs from a table and compute coef/SE yourself. 229 of 3187 t -stats (7.2%) collapse to exactly $t = 2.0$ — a pure rounding artifact.



No p-hacking in this simulation. True t -stats drawn from a smooth distribution with no mass point anywhere. Rounding to 1–3 decimal places (mix matching real APE papers) creates the spikes at integer ratios.

Coefficients and SEs in published papers are rounded. Dividing a rounded coefficient by a rounded SE piles t up at integer ratios: 1, 2, 3.

I simulated 3,500 regressions *with no p-hacking*. Same heaping.

Removing 68 exact- $t = 2$ rows dropped the Brodeur ratio from 1.52 to **1.02**. Brodeur et al. have since acknowledged the issue.

Jason Fletcher and Gorkem Ozer caught it. I did not.

This is the experience that led me to write /blindspot — I had started to wonder if I was going to miss the obvious.

Even zero discretion leaves implementation risk

Independent Hallucination Errors. A second experiment.

- ▶ Replicate Dias & Fontes (2024, *AEJ: Economic Policy*) on the same CAPS data.
- ▶ **96 zero-discretion agents.** Every decision prescribed in advance.
- ▶ Vary only what the principal might otherwise vary silently: the package and the covariate set.

Six implementations of the same estimator. Sixteen specifications. Same data, same method, same controls. They should agree.

Six packages implement the same estimator differently

Package	Language	Author
did	R	Callaway & Sant'Anna
ddml	R	Ahrens, Hansen, Schaffer, Wiemann
csdid	Stata	Rios-Avila
csdid2	Stata	Rios-Avila
differences	Python	Dionisi
diff-diff	Python	Gerber

All using doubly robust + not-yet-treated controls + universal base period.

Identical specification across all six.

With no covariates, all six packages agree exactly

did (R)
0.311

ddml (R)
0.311

csdid (Stata)
0.311

csdid2 (Stata)
0.311

differences (Py)
0.311

diff-diff (Py)
0.311

ATT ≈ 0.311
no covariates

The estimator itself is implemented correctly.

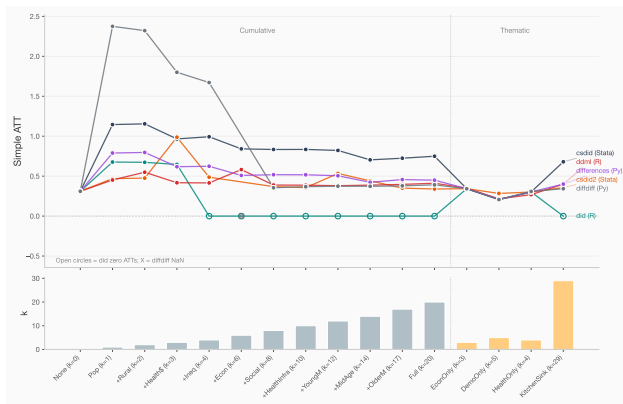
Add one covariate, and the agreement breaks

One covariate: `poptotaltrend` — **municipal population** $\times$ **year**.

Package	ATT
ddml (R)	0.45
csdid2 (Stata)	0.45
did (R)	0.53
differences (Python)	0.67
csdid (Stata)	1.15
diff-diff (Python)	2.38

Same data. Same estimator. Same single covariate. A 5 $\times$ spread.

The disagreement fans out as covariates accumulate



Each line is one of six packages, plotted across 16 specifications. All start at the same point and then fan out.

The smoking gun: remove that one variable

Run specs using only economics, demographics, or health covariates, excluding `poptotaltrend`.

All six packages agree to within 0.007.

One variable caused the entire 5× spread.

The matrix inversion is the easy part

The propensity-score logit's design matrix X has a column `poptotaltrend` (population $\times$ year). For a 50,000-person municipality in 2010, that single cell is **100,500,000**. The variable lives on the order of 10^{10} .

The other covariates live on the order of 0.01 to 16,000. Mixed in the same X , the condition number of $X'X$ explodes to roughly 3.6×10^9 .

Below machine epsilon (2.22×10^{-16}): the computer literally cannot distinguish the columns at 64-bit precision. Inverting $X'X$ — which every propensity-score logit needs — produces garbage.

But this is just a logistic regression. If $X'X$ is hard to invert, shouldn't *all six* packages have a hard time with it?

The answer turns out to be no.

Six “logistic regressions” are not the same logistic regression

Package	Lang	Routine called	Optimizer
did	R	<code>glm(probit)</code>	IRLS
ddml	R	<code>glmnet Lasso (DR) + glm(probit) trim</code>	L1 coord. descent + IRLS
csdid	Stata	<code>probit, iterate(1000) difficult + drimp</code>	Newton–Raphson
csdid2	Stata	<code>probit, difficult ltol(0) tol(1e-5)</code>	Newton–R., relaxed tol.
differences	Py	<code>statsmodels.Probit().fit()</code>	statsmodels Newton
diff-diff	Py	<code>statsmodels.Probit() + CallawaySantAnna</code>	statsmodels + internal

Same name on the API. Different optimizer. Different singular-matrix behavior. The user does not see this.

Each row is the actual call inside the package, recovered from the source.

Is this wrong?

What we found:

- ▶ `did` *silently* returns all-zero ATTs without raising
- ▶ `csdid` requires `iterate(1000)` difficult to nominally converge
- ▶ `csdid2` runs with relaxed tolerance: “stabilized but not formally converged”
- ▶ `ddml`’s Lasso auto-standardizes inside the learner
- ▶ `statsmodels` converges silently — possibly to wrong values

Honest answer: no individual choice is indefensible. Each package author made a reasonable engineering call. But the choice is invisible to the user. The documentation says “logistic regression.”

The new researcher degrees of freedom: sort into language → package → hidden algorithmic action. Three layers; only the first appears in methods.

The fix: z-score the covariates, or divide `poptotaltrend` by 10^6 . The ill-conditioning is the trigger; remove the trigger and most of the divergence collapses.

Hidden actions become detectable only when independent agents replicate one another.

Other issues found

1. Cohort-specific propensity scores. CS fits a separate propensity-score logit *per treatment cohort*. With 500 treated municipalities split across cohorts, each PS regression sees only N_c observations. If k is too large for that N_c , you get erratic perfect separation — wrapped inside the logistic, independent of the algorithmic heterogeneity above.

2. Silent covariate dropping. Some logistics, when they cannot estimate the full X , silently drop the covariates and regress D on a constant only — without telling the user. The package hides the propensity-score coefficient output and often does not store the propensity scores after estimation. The researcher — and especially Claude in automation mode — may not realize what just happened.

Hidden actions become detectable only when independent agents replicate one another.

Note: `diff-diff` (Gerber) has since been updated; the Round-4-era limitation has been addressed.

Agents make expensive verification routines cheap enough to run

AI did not magically know the answer.

The workflow made the discrepancy visible:

- ▶ many independent agents
- ▶ multiple languages
- ▶ six packages
- ▶ forced comparison to a common target

Agents are useful because they make expensive verification routines cheap enough to run.

5

What To Do Monday Afternoon

Start with audits whose outputs you can judge

Start with audits whose outputs you can inspect:

1. **Citation audit:** Does my literature review accurately represent each cited paper?
2. **Repo audit:** Can a fresh agent install, run, and map this replication package?
3. **Code audit:** Can a fresh agent reimplement my analysis in another language?
4. **Referee audit:** What would a skeptical reader attack first?
5. **Deck audit:** Does my presentation make one clear claim per slide?

Do not begin with the task you least understand. Begin where you can judge the output.

A good assignment makes the output inspectable

Five assignment rules:

1. Give it a folder with the relevant files.
2. State the role: producer, auditor, adversary, replicator.
3. Make the output concrete: table, memo, deck, diff, error list.
4. Force source-grounding: quote line numbers, cite files, compare outputs.
5. Use fresh agents for verification.

The unit of progress is not a clever prompt. It is a repeatable workflow.

Homework: build a skill, do not just borrow one

If you make only one skill after this talk, make it a citation-audit skill. Pick a repeating task and turn it into a procedure your agent can run.

Why build instead of borrow. Ethan Mollick has noted that open source for skills is not the same as open source for code. A skill carries the author's prompts, assumptions, and failure modes. The audit surface is fuzzier than source. You inherit what you install.

Why I show you mine anyway. The point is the pattern, not the prompt. Fork, rewrite, replace — the right thing to copy is the design move, not the trust.

Help me create a citation-audit skill for my papers.

It should: identify each cited claim;

spawn one agent per cited paper; compare my claim to source;

return accurate / overstated / missing / unclear.

Ask me questions and iterate before writing the final skill.

Budget for the real tool, not the demo tier

Agentic research work is not the same as asking a chatbot one question.

For long-running tasks, you want the strongest available model and enough usage headroom:

- ▶ Agents read many files, write code, run it, hit errors, inspect output, and try again.
- ▶ Smaller or capped models may look fine in demos but time out once you depend on the workflow.
- ▶ The relevant comparison is not free vs. paid. It is whether the agent can finish the job.

As of this talk: **Codex is roughly half the monthly price of Claude Max** — about \$100 vs. \$200.

If agents become part of your research process, buy enough model to avoid making the workflow brittle.

When agents fail, ask where the workflow failed

When an LLM underdelivers, there are two possible explanations:

Model failure

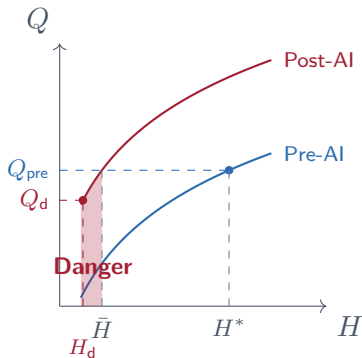
The model genuinely lacks the capability, knowledge, or reasoning required.

Workflow failure

The user assigned the wrong task, gave the wrong context, allowed too much discretion, or failed to verify.

In my experience, workflow failure is usually the first suspect.

Deskilling is also a verification risk



AI can reduce the time and attention used in production.

That is the gain. But skill often came from exactly that time and attention.

- ▶ less production practice can mean less verification skill
- ▶ verification depends on accumulated judgment
- ▶ replication costs are falling toward zero
- ▶ outside scrutiny and penalties may rise

Be cognizant of the deskilling margin.

The optimistic path is more verification per unit of attention

Deskilling is possible. Lost attention is possible.

But the alternative is also possible:

- ▶ more citation claims checked against sources
- ▶ more replication packages actually run
- ▶ more code implemented twice
- ▶ more papers exposed to skeptical readers before submission
- ▶ more talks shaped around audience, not slide labor

The optimistic path is not less human judgment. It is more verification per unit of human attention.

The new skill is knowing what must be verified

The scarce skill is not typing prompts.

The scarce skill is knowing:

- ▶ what to delegate
- ▶ what to constrain
- ▶ what to audit independently
- ▶ what disagreement means
- ▶ when the final judgment must be human

AI agents make production cheap. The researcher's job is to make verification systematic.

Setup links are now an attack surface

Agent workflows often ask you to copy commands into a terminal.

That changes phishing:

- ▶ cloned sites can imitate `claude.ai`, GitHub, or package docs
- ▶ a copy button can hide a dangerous install command
- ▶ agents can browse, compare domains, inspect repos, and explain commands
- ▶ do not paste terminal commands you do not understand

Use agents defensively too: ask them to verify the source before you trust the setup.

Recent threat reports: Google Threat Intelligence on AI-assisted social engineering; Cloud Security Alliance on indirect prompt injection.

**Production is cheap.
Verification is the craft.**

Scott Cunningham

Baylor University · causalinf.substack.com